

SPRAWOZDANIE - ZADANIE 1

Build lokalny

```
Victus@DESKTOP-L681EK6 MINGW64 ~/VSCProjects/Studia VSC/Semestr 6/Chmura/ZADANIE1 (lab6)
$ docker build -t mickup177/weather-app:v5 .
[*] Building 186.3s (18/18) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring Dockerfile: 1.56kB
=> resolve image config for docker-image://docker.io/docker/dockerfile:1
=> [auth] docker/dockerfile:pull token for registry-1.docker.io
=> CACHED docker-image://docker.io/docker/dockerfile:1@sha256:2780b5c3bab67f1f76c781868de469442999ed1a8d7992a5efdf2cfc8e3d769
=> => resolve docker.io/docker/dockerfile:1@sha256:2780b5c3bab67f1f76c781868de469442999ed1a8d7992a5efdf2cfc8e3d769
=> [internal] load metadata for docker.io/library/golang:1.21-alpine
=> [auth] library/golang:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 116B
=> [builder 1/6] FROM docker.io/library/golang:1.21-alpine@sha256:2414835b886e3c42b99654c8b26e6f5b1b1598888d65fd83c7f499552ff4dc94
=> => resolve docker.io/library/golang:1.21-alpine@sha256:2414835b886e3c42b99654c8b26e6f5b1b1598888d65fd83c7f499552ff4dc94
=> [internal] load build context
=> => transferring context: 29B
=> CACHED [builder 2/6] RUN apk add --no-cache ca-certificates upx
=> CACHED [builder 3/6] WORKDIR /src
=> CACHED [builder 4/6] COPY main.go .
=> [builder 5/6] RUN CGO_ENABLED=0 GOOS=linux go build -ldflags="-s -w -extldflags '-static'" -trimpath -buildvcs=false -o weather-app main.go && upx 182.6s
=> [builder 6/6] RUN echo "app:x:10001:10001::/* > /etc/passwd.min
=> [stage-1 1/3] COPY --from=builder /etc/passwd.min /etc/passwd
=> [stage-1 2/3] COPY --from=builder /etc/ssl/certs/ca-certificates.crt /etc/ssl/certs/
=> [stage-1 3/3] COPY --from=builder /src/weather-app /weather-app
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:21d54fb0a9e3793c881f86dc217732ba57c8d6897722aed1e539e8094cb1dc6
=> => exporting config sha256:4a8013569371305c3b4d70d566283111433b65340823e416535e7278bd3a
=> => exporting attestation manifest sha256:8368b733ae7635c1f8ca7866a1828e764bae4dcea8afde82089429c27a3be46
=> => exporting manifest list sha256:3f414de21ce6598cf854f6cfdea4e42873a9d2df2928749cd96bcecf695fffd8
=> => naming to docker.io/mickup177/weather-app:v5
=> => unpacking to docker.io/mickup177/weather-app:v5
View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/hxt2x2wm51b1o4xxvm40afhd1
```

Potwierdzenie działania, wielkość obrazu oraz logi:

Potwierdzenie poprawnego uruchomienia aplikacji. W logach widoczne są wymagane dane: data startu, imię i nazwisko autora oraz port TCP (8080), na którym nasłuchuje server.

Widok listy uruchomionych kontenerów. Status (healthy) potwierdza, że mechanizm sprawdzania sprawności aplikacji działa poprawnie i aplikacja reaguje na zapytania wewnętrzne.

```
Victus@DESKTOP-L681EK6 MINGW64 ~/VSCProjects/Studia VSC/Semestr 6/Chmura/ZADANIE1 (lab6)
$ docker run -d --name weather-v5 -p 8081:8080 mickup177/weather-app:v5
5565f1e423cb9385c885d2ca451cb43bf7f598dfb68585fbd81f67bfbb09a324

Victus@DESKTOP-L681EK6 MINGW64 ~/VSCProjects/Studia VSC/Semestr 6/Chmura/ZADANIE1 (lab6)
$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
5565f1e423cb   mickup177/weather-app:v5           "/weather-app"          5 seconds ago Up 5 seconds (health: starting) 0.0.0.0:8081->8080/tcp, [::]:8081->8080/tcp   weather-v5
f29a254f1dfe   mickup177/weather-app:v4           "/weather-app"          15 minutes ago Up 15 minutes (healthy)      0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp   weather-v4
a77faed99cd8   noby/buildkit:buildx-stable-1       "/usr/bin/buildkitd-..." 13 days ago   Up 13 days                                     buildx_buildkit_testbuilder28

Victus@DESKTOP-L681EK6 MINGW64 ~/VSCProjects/Studia VSC/Semestr 6/Chmura/ZADANIE1 (lab6)
$ docker logs weather-v5
START: [2026-05-04T09:19:18Z] Autor: Michał Kupidura, Port: 8080
```

Dzięki zastosowaniu obrazu bazowego scratch oraz kompresji UPX, finalny rozmiar obrazu wynosi zaledwie ok. 2.38 MB. Jest to wynik optymalny pod kątem konkursu na najmniejszy obraz.

```
Victus@DESKTOP-L681EK6 MINGW64 ~/VSCProjects/Studia VSC/Semestr 6/Chmura/ZADANIE1 (lab6)
$ docker images mickup177/weather-app:v5
IMAGE                                ID                                DISK USAGE  CONTENT SIZE  EXTRA
mickup177/weather-app:v5            3f414de21ce6                     4.88MB      2.38MB       U
```

Raport bezpieczeństwa Docker Scout

Analiza podatności wykazała 0 zagrożeń typu Critical oraz High. Obraz jest bezpieczny, ponieważ nie zawiera zbędnych bibliotek systemowych ani powłoki shell.

```
Victus@DESKTOP-L681EK6 MINGW64 ~/VSCProjects/Studia VSC/Semestr 6/Chmura/ZADANIE1 (zadaniel)
$ docker scout quickview mickupi77/weather-app:v5
  i New version 1.20.4 available (installed version is 1.20.0) at https://github.com/docker/scout-cli
  v Image stored for indexing
  v Indexed 0 packages

  i Base image was auto-detected. To get more accurate results, build images with max-mode provenance attestations.
  Review docs.docker.com / for more information.

Target | mickupi77/weather-app:v5 | 0C | 0H | 0M | 0L
digest | 3f414de21ce6

What's next:
  Include policy results in your quickview by supplying an organization → docker scout quickview mickupi77/weather-app:v5 --org <organization>
```

Build na wiele platform.

Widok repozytorium po wykonaniu buildx. Obraz posiada wsparcie dla architektur

amd64 oraz arm64, co pozwala na jego uruchomienie na serwerach x86 oraz

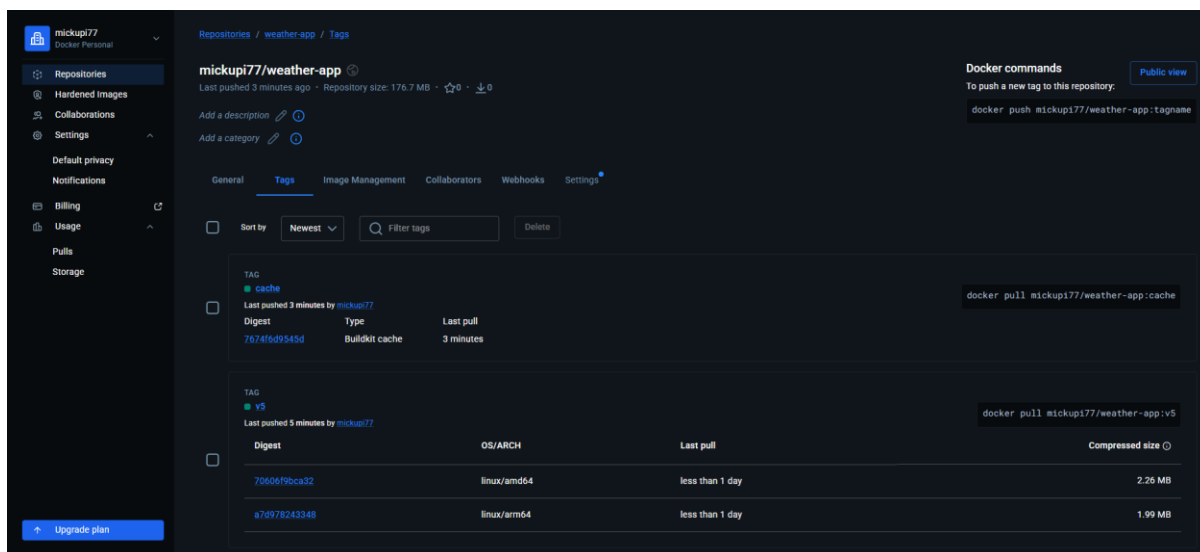
komputerach z procesorami Apple M1/M2.

```
Victus@DESKTOP-L681EK6 MINGW64 ~/VSCProjects/Studia VSC/Semestr 6/Chmura/ZADANIE1 (zadaniel)
$ docker buildx build --platform linux/amd64,linux/arm64 \
  -t mickupi77/weather-app:v5 \
  --cache-to type=registry,ref=mickupi77/weather-app:cache,mode=max \
  --cache-from type=registry,ref=mickupi77/weather-app:cache \
  --push .
[+] Building 1118.2s (33/33) FINISHED                                docker-container:testbuilder2
=> [internal] load build definition from Dockerfile                  0.1s
=> => transferring dockerfile: 1.09kB                                0.0s
=> resolve image config for docker-image://docker.io/docker/dockerfile:1      1.2s
=> [auth] docker/dockerfile:pull token for registry-1.docker.io      0.0s
=> CACHED docker-image://docker.io/docker/dockerfile:1@sha256:2788b5c3bab67f1f76c781868de469442999ed1a8d7992a5efd2cfcfc9e3d769 0.1s
=> => resolve docker.io/docker/dockerfile:1@sha256:2788b5c3bab67f1f76c781868de469442999ed1a8d7992a5efd2cfcfc9e3d769 0.0s
=> [linux/arm64 internal] load metadata for docker.io/library/golang:1.21-alpine 1.0s
=> [linux/amd64 internal] load metadata for docker.io/library/golang:1.21-alpine 1.8s
=> [auth] library/golang:pull token for registry-1.docker.io          0.0s
=> [internal] load .dockerignore                                       0.1s
=> => transferring context: 116B                                         0.0s
=> ERROR importing cache manifest from mickupi77/weather-app:cache    0.0s
=> [internal] load build context                                       0.0s
=> => transferring context: 29B                                           0.0s
=> CACHED [linux/amd64 builder 1/6] FROM docker.io/library/golang:1.21-alpine@sha256:2414035b086e3c42b99654c8b26e6f5b1b1598080d65fd83c7f499552ff4dc94 0.1s
=> => resolve docker.io/library/golang:1.21-alpine@sha256:2414035b086e3c42b99654c8b26e6f5b1b1598080d65fd83c7f499552ff4dc94 0.1s
=> [auth] mickupi77/weather-app:pull token for registry-1.docker.io    0.0s
=> CACHED [linux/amd64 builder 1/6] FROM docker.io/library/golang:1.21-alpine@sha256:2414035b086e3c42b99654c8b26e6f5b1b1598080d65fd83c7f499552ff4dc94 0.1s
=> => resolve docker.io/library/golang:1.21-alpine@sha256:2414035b086e3c42b99654c8b26e6f5b1b1598080d65fd83c7f499552ff4dc94 0.1s
=> [linux/amd64 builder 2/6] RUN apk add --no-cache ca-certificates upx 2.8s
=> [linux/arm64 builder 2/6] RUN apk add --no-cache ca-certificates upx 7.4s
=> [linux/amd64 builder 3/6] WORKDIR /src                               0.1s
=> [linux/amd64 builder 4/6] COPY main.go                             0.1s
=> [linux/amd64 builder 5/6] RUN CGO_ENABLED=0 GOOS=linux go build -ldflags="-s -w -extldflags '-static'" -trimpath -buildvcs=false -o weather-app main.go 593.0s
=> [linux/arm64 builder 3/6] WORKDIR /src                               0.1s
=> [linux/arm64 builder 4/6] COPY main.go .                             0.1s
=> [linux/arm64 builder 5/6] RUN CGO_ENABLED=0 GOOS=linux go build -ldflags="-s -w -extldflags '-static'" -trimpath -buildvcs=false -o weather-app main.go 998.1s
=> [linux/amd64 builder 6/6] RUN echo "app:x:10001:10001:/:/etc/passwd min 0.3s
=> [linux/amd64 stage-1 1/3] COPY --from=builder /etc/passwd.min /etc/passwd 0.1s
=> [linux/amd64 stage-1 2/3] COPY --from=builder /etc/ssl/certs/ca-certificates.crt /etc/ssl/certs/ 0.1s
=> [linux/amd64 stage-1 3/3] COPY --from=builder /src/weather-app /weather-app 0.1s
=> [linux/arm64 builder 6/6] RUN echo "app:x:10001:10001:/:/etc/passwd min 0.3s
=> CACHED [linux/amd64 stage-1 1/3] COPY --from=builder /etc/passwd.min /etc/passwd 0.0s
=> CACHED [linux/amd64 stage-1 2/3] COPY --from=builder /etc/ssl/certs/ca-certificates.crt /etc/ssl/certs/ 0.1s
=> [linux/arm64 stage-1 3/3] COPY --from=builder /src/weather-app /weather-app 0.1s
=> => exporting to image                                                 10.6s
=> => exporting layers                                                  0.6s
=> => exporting manifest sha256:70686f9bca321724bfff8df2e386cc36f12c1c2e88c093eff4ff9594fad381f26 0.0s
=> => exporting config sha256:b0ccf82377724724cabcbe8f6c824b8c398336771d8e3804d598747f9dd427 0.0s
=> => exporting attestation manifest sha256:bbe0bc3f47883be4f5dd0d547a0f3ba4dd68635da8e5ba31976b4392e94b5527e55 0.1s
=> => exporting manifest sha256:a7d978203348a0b9f422d9286d7b832c977561cf586f663fd25381e7b7b1c1 0.0s
=> => exporting config sha256:bfa485cc531b756e60d26b5732d6e831a6e57dab4617f919ee9f060ff4b7fc145 0.0s
=> => exporting attestation manifest sha256:1cb9d656e01b6d01f4534042861fe39107b64ddba6219ad0da4d812846898b 0.1s
=> => exporting manifest list sha256:b6de49693344eb271c80f8ad9d95635bfa72b3d649ca879b880432a26a193c97 0.1s
=> => pushing layers                                                    5.4s
=> => pushing manifest for docker.io/mickupi77/weather-app:v5@sha256:b6de49693344eb271c80f8ad9d95635bfa72b3d649ca879b880432a26a193c97 4.2s
=> => exporting cache to registry                                       114.5s
=> => preparing build cache for export                                  10.8s
=> => sending cache export                                              183.7s
=> writing layer sha256:1338436c54215e4050e365259bfa7738c7d7d55373456f771a1cd72464f797b14 1.7s
=> writing layer sha256:12d281ef8c7684b6c35a67d495584f77369bea6a26e622de674633b16db62a9 1.7s
=> writing layer sha256:109f275a11062b3d97563161c3d79bc6189988ea99368d4b595a1d2801b2896 1.7s
=> writing layer sha256:111ecd6efba15b3c0f9496c52609231b4218a5d20f2b24fc0b3a150724cab096 3.0s
=> writing layer sha256:161d8206e5f132bed822ea3a7feb25f0c98364706b5d6892a139ea81bfa5cce7 26.2s
=> writing layer sha256:774ef86a7275b852f9c393fac9801a0153330f2de16587798349ae33d72d02a 1.4s
=> writing layer sha256:204534d3e1f9b9b2d886d71613061cd1519088977f4fc922bb318ac801a8d1f6 1.4s
=> writing layer sha256:2a6022646f09ee78a83ef4abd0f5af04071b6563cf16a18e00fb2dcfe63ca0a3 91.2s
=> writing layer sha256:41db7493d1c6f3726428d119962e3862c149a20bb0b8f0c36e7282d015e099 1.3s
=> writing layer sha256:4579088f8580d429ec087d092329191089711942d9380d060c8d9bd24c0352c 1.0s
=> writing layer sha256:444f4b700e450461cfa02571a080b9a8dc1e0cd5577484ad675e68dc38e8acc1 0.7s
=> writing layer sha256:54bf7032e096c2c74637ad1f80bd6d1345bc9fabb1531fd8894aa8a9af0 95.5s
=> writing layer sha256:643533f23b9939deddc3882aa939190dd40a3e388dc38417587394d9fbd2d0f5 0.2s
=> writing layer sha256:660d870c6bd029ba8f8c98dfce578d9e3cf141bb2ec83ba94d237523274a67bba 0.2s
=> writing layer sha256:690e8786737b8441990047e169b892933e9006bdcdbcd52ab7a356945477a4d 7.1s
=> writing layer sha256:698a08e286508cb27356cf4f957a31c2f38e2fd5b88e5c0b183de8d213e4c546 1.1s
=> writing layer sha256:701ed4299cb709339246a28a4c56254c6d74e6fba92831ddd96896e7ce74b151 1.1s
=> writing layer sha256:76891dd181e59c8d8959f7510b9944ab1e6be9a8de4b551cc89521cdc994d2 20.8s
=> writing layer sha256:76891dd181e59c8d8959f7510b9944ab1e6be9a8de4b551cc89521cdc994d2 0.1s
=> writing layer sha256:774fe86d775b8852f9c393fac9801a0153330f2de16587798349ae33d72d02a 0.1s
=> writing layer sha256:8b8d41246191772d65e82f304c55b6f7d0364b488a9e82d7b3347cde7a8732 1.1s
=> writing layer sha256:c683fedfae6ed8a4f5f7cbb6a7b6f1c1ec3d86fea8cb9e5ba2e5e6673fde9f6 4.1s
=> writing layer sha256:d98a09e786580cb27f36cf4f957a31c2f39e4f03b08e3c0b834e8d713e4c940 1.1s
=> writing layer sha256:701c46299cb709339246a28a4c56254c6d74e6fba92831ddd96896e7ce74b151 28.8s
=> writing layer sha256:76891dd181e59c8d8959f7510b9944ab1e6be9a8de4b551cc89521cdc994d2 0.1s
=> writing layer sha256:774fe86d775b8852f9c393fac9801a0153330f2de16587798349ae33d72d02a 0.1s
=> writing layer sha256:8b8d41246191772d65e82f304c55b6f7d0364b488a9e82d7b3347cde7a8732 1.1s
=> writing layer sha256:c683fedfae6ed8a4f5f7cbb6a7b6f1c1ec3d86fea8cb9e5ba2e5e6673fde9f6 4.1s
=> writing layer sha256:db4c3392d42466a9701b90b7a05f5ee517040d2e943a927a7239d6565c0bc20b50 2.8s
=> writing layer sha256:e495eface5cc12777f452389e1da15202c37ec08ba624f12f841b5c98a470857 1.2s
=> writing config sha256:13499bccc06b09eb78a9c77483c6819a7629d2f0e583c58c6986ad1f088a93 1.2s
=> writing cache image manifest sha256:7674f6d09545daf0821004ea35fa530cecf7cb85c149e2f1ec83f79b28b55e9ab 1.0s
=> [auth] mickupi77/weather-app:pull,push token for registry-1.docker.io 0.0s
=> [auth] mickupi77/weather-app:pull,push token for registry-1.docker.io 0.0s

> importing cache manifest from mickupi77/weather-app:cache:
-----

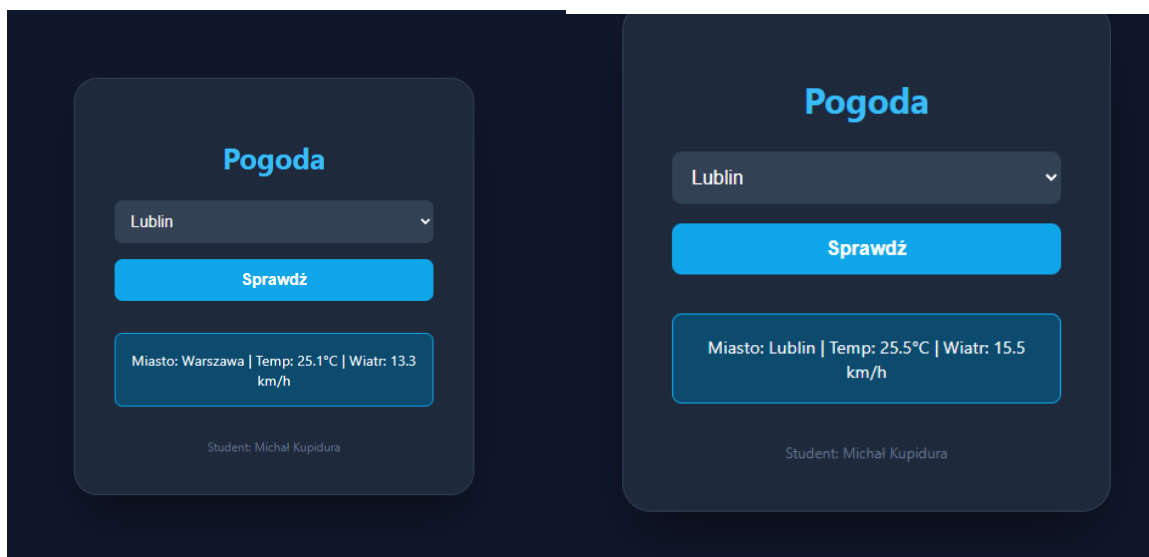
View build details: docker-desktop://dashboard/build/testbuilder2/testbuilder20/ppbaftrfnp96a2argliurstu
```

Potwierdzenie na DockerHub



Działanie aplikacji pod adresem localhost:8080

Widok strony głównej serwowanej z kontenera. Wybrano miasto Lublin, a aplikacja poprawnie pobrała i wyświetliła dane z API Open-Meteo. Całość ostylowana w trybie "Dark Mode" za pomocą inline CSS.



Podsumowanie

Zastosowałem obraz bazowy scratch w połączeniu z agresywną kompresją binarną UPX, co pozwoliło mi na redukcję rozmiaru obrazu z około 300 MB (charakterystycznych dla standardowych obrazów Go) do zaledwie 2.4 MB. Udowadnia to, że wybór odpowiedniego języka kompilowanego statycznie jest kluczowy w architekturze mikroserwisowej, gdzie minimalizacja zajmowanego miejsca bezpośrednio przekłada się na szybkość dystrybucji i koszt utrzymania infrastruktury.

Wykorzystałem warstwę scratch, aby drastycznie zredukować tak zwany *attack surface* (powierzchnię ataku). Mój obraz nie zawiera powłoki bash, menedżerów pakietów ani zbędnych bibliotek systemowych, dzięki czemu skanery takie jak Docker Scout nie wykazują żadnych podatności, co potwierdza raport wskazujący 0 luk typu Critical oraz High. Dodatkowo, świadomie skonfigurowałem uruchamianie procesu jako użytkownik non-root, co stanowi krytyczną warstwę zabezpieczającą system hosta w przypadku wystąpienia błędu w samej aplikacji.

Zadbałem również o pełną wieloplatformowość zgodnie ze standardami Cloud-Native. Dzięki wykorzystaniu narzędzia docker buildx, moja aplikacja stała się w pełni przenośna – ten sam obraz działa bez przeszkód zarówno na serwerach chmurowych (architektura amd64), jak i na nowoczesnych laptopach czy urządzeniach IoT (architektura arm64). Takie podejście jest obecnie absolutnym standardem w nowoczesnym cyklu wytwarzania oprogramowania oraz profesjonalnych procesach CI/CD.

Na koniec zaimplementowałem natywny mechanizm HEALTHCHECK bez konieczności użycia zewnętrznych narzędzi typu curl. Takie rozwiązanie pozwala silnikowi Docker na w pełni automatyczne i wydajne monitorowanie stanu zdrowia aplikacji bezpośrednio z wewnątrz kontenera. W razie ewentualnej awarii, mechanizm ten umożliwia automatyczny restart kontenera przez system zarządzający, co znacząco podnosi odporność i niezawodność całego wdrożonego rozwiązania.